

程式語言(一)

Week 1：課程介紹與程式思維

歡迎來到程式設計的世界！本週我們將一起探索 C 語言的核心概念,建立紮實的程式思維基礎。



 課程概覽

課程資訊

核心內容

深入學習 C 語言基礎,掌握程式設計的本質與邏輯思維。

課程名稱

程式語言(一)

主要語言

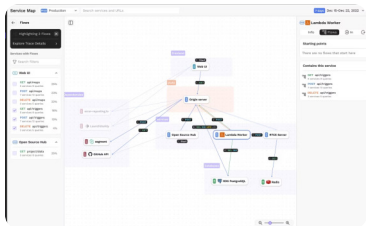
C 語言基礎

每週安排

講課 4 小時

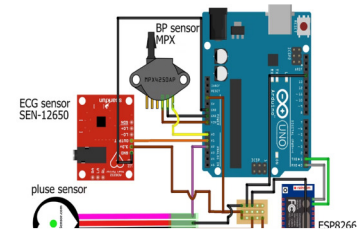
實驗 1 小時

為什麼學 C 語言？



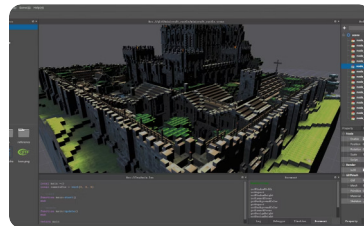
作業系統核心

Linux、Windows、macOS 等主流作業系統都使用 C 語言開發核心元件,是系統程式設計的基石。



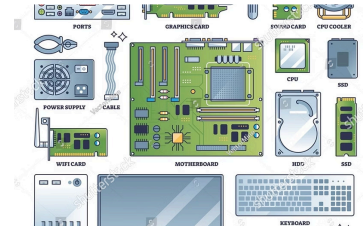
嵌入式系統

Arduino、物聯網裝置等嵌入式系統的主力開發語言,控制硬體的最佳選擇。



遊戲引擎底層

高效能遊戲引擎如 Unreal、Unity 的底層架構,追求極致效能的關鍵技術。



底層運作理解

深入了解電腦如何運作,掌握記憶體管理與硬體控制的核心原理。

C 語言的歷史與設計哲學

1972 年誕生

Dennis Ritchie 在貝爾實驗室開發,用於 UNIX 系統開發。

1

2

簡潔而強大

最小化抽象層次,直接操作記憶體,提供底層控制能力。

高效執行

編譯成機器碼,執行速度快且具備優異的可移植性。

3

4

底層控制

精確管理記憶體與硬體資源,是系統程式設計的理想選擇。

C 語言與其他語言的差異

型別系統

靜態型別,編譯時檢查,避免執行時錯誤。不同於 JavaScript 的動態型別。

記憶體管理

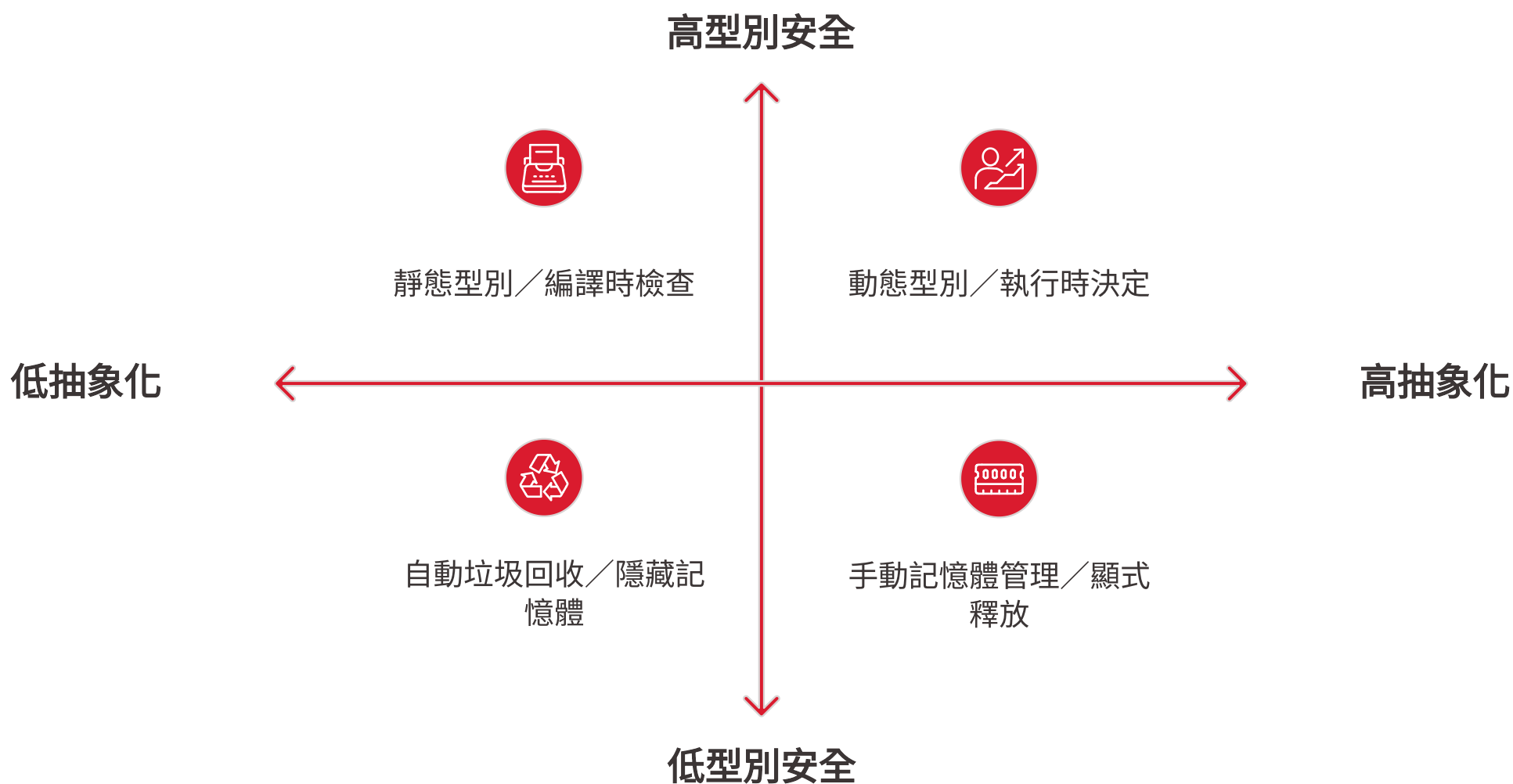
手動分配與釋放記憶體,程式設計師完全掌控,無自動垃圾回收機制。

指標操作

顯式使用指標,直接存取記憶體地址,提供強大的底層控制能力。

執行方式

編譯成機器碼後執行,效能優異,不同於直譯語言的即時執行。



學習 C 語言的優勢



執行速度快

編譯成高效機器碼,執行速度接近組合語言,適合效能關鍵應用如即時系統、高頻交易系統等。



深入底層理解

透過指標、記憶體管理等概念,深刻理解電腦底層運作機制,建立紮實的計算機科學基礎。



廣泛應用領域

系統程式、嵌入式裝置、遊戲引擎、資料庫系統等眾多關鍵領域的首選開發語言。



嚴謹程式思維

培養細緻的記憶體管理意識與安全程式設計習慣,為日後學習其他語言打下堅實基礎。

Jan 2026	Jan 2025	Change	Programming Language		Ratings	Change
1	1		 Python		22.61%	-0.68%
2	4	▲	 C		10.99%	+2.13%
3	3		 Java		8.71%	-1.44%
4	2	▼	 C++		8.67%	-1.62%
5	5		 C#		7.39%	+2.94%
6	6		 JavaScript		3.03%	-1.17%
7	9	▲	 Visual Basic		2.41%	+0.04%
8	8		 SQL		2.27%	-0.14%
9	11	▲	 Delphi/Object Pascal		1.98%	+0.19%
10	18	▲	 R		1.82%	+0.81%
11	32	▲	 Perl		1.63%	+1.14%

TIOBE 程式語言排行榜

TIOBE Index 是全球最具權威的程式語言流行度指標。C 語言穩居第二名，尤其在嵌入式系統與物聯網領域依然不可取代。



TIOBE Index – TIOBE

Let op! Internet Explorer wordt niet meer ondersteund. Hierdoor kan de website mogelijk niet goed functioneren, gebruik een alternatieve browser om optimaal gebruik te maken van deze website. [Klik hier om een alternatieve browser te downloaden.](#)



五大核心能力

本課程將幫助你建立完整的程式設計能力,從理解到實作,從分析到創造。



讀懂並分析程式碼

能夠閱讀他人撰寫的程式碼,理解其邏輯結構與設計意圖。



追蹤程式執行流程

掌握程式執行的每個步驟,理解變數變化與函式呼叫過程。



找出並修正程式錯誤

具備除錯能力,能夠識別語法錯誤、邏輯錯誤與執行時錯誤。



判斷程式碼品質與風格

評估程式碼的可讀性、維護性與效率,培養良好的程式設計美學。



能夠獨立撰寫程式

從問題分析到程式實作,完整掌握程式開發的全流程。

AI 時代的程式學習挑戰

AI 是工具非替代

AI 可協助生成程式碼,但無法取代你的判斷力與創造力。

培養程式語感

理解程式本質比單純寫碼更重要,建立對優質程式碼的直覺。

避免盲目模仿

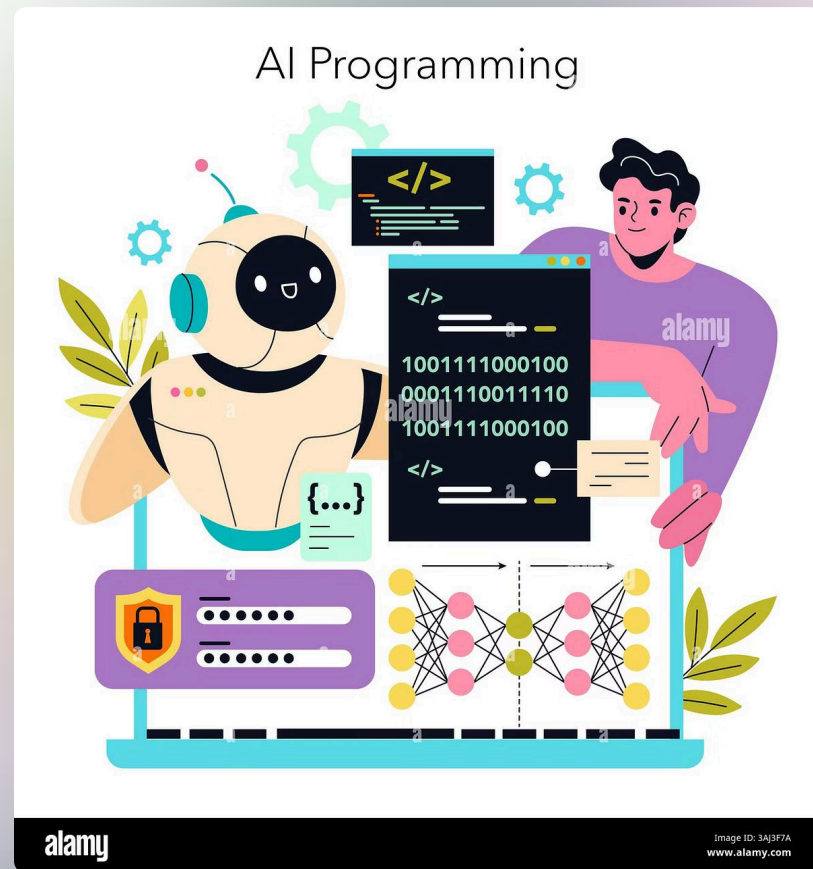
「Vibe Coding」無法帶來真正成長,必須深入理解底層邏輯。

駕馭 AI 工具

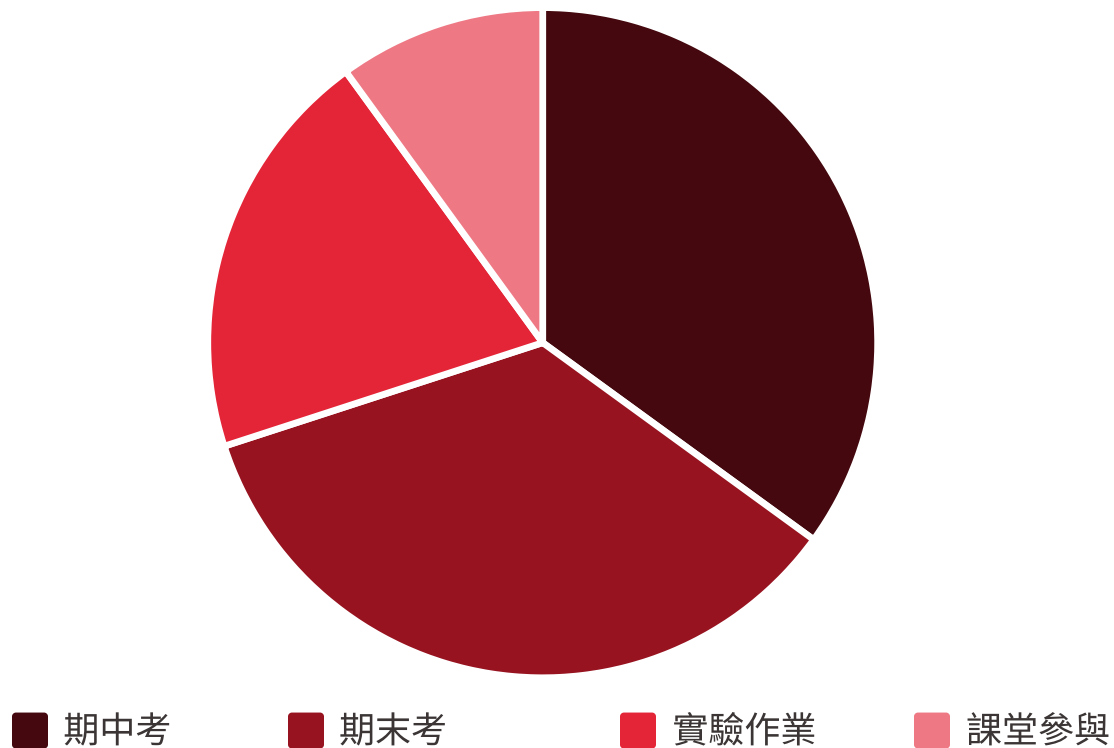
只有理解底層原理,才能有效運用 AI 工具提升開發效率。

重新定義學習價值

在 AI 工具如 ChatGPT、Copilot 普及的時代,程式學習的核心價值已經改變。單純模仿程式碼已不足夠,真正的能力在於理解、判斷與創新。



評分方式



評量說明

- **期中考與期末考(各 35%)**:測驗對 C 語言核心概念的理解與應用能力
- **實驗作業(20%)**:透過實作練習鞏固學習成果,培養獨立解決問題的能力
- **課堂參與(10%)**:積極參與討論與提問,展現學習熱忱與思考深度

持續努力與積極參與是學習程式設計的關鍵!

C 語言程式範例:Hello World

讓我們從經典的 Hello World 程式開始,這是每位程式設計師的第一步。

```
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

01

引入標準輸入輸出函式庫

`#include <stdio.h>` 讓我們可以使用 `printf` 函式

03

輸出文字

`printf("Hello, World!\n")` 在螢幕上顯示文字,`\n` 代表換行

02

定義主函式

`int main()` 是程式的進入點,所有 C 程式都從這裡開始執行

04

結束程式

`return 0` 表示程式正常結束,回傳 0 給作業系統